

SPRINT 2

Inventário de Ativos e Vulnerabilidades

Material de apoio para apresentação e estudo

Aluno: Matheus Moreli Durão Silva

Matrícula: 12621CBS217

E-mail: matheus.moreli@ufu.br

Curso: Cibersegurança - Bacharelado - Noturno

Sumário

1	O que mudou da Sprint 1 para a Sprint 2	2
2	Organização simples do código	2
3	Classes e Enum	2
3.1	S2_01 e S2_02 – Boas práticas, classes e funções	2
4	CRUD de ativos e vulnerabilidades	3
4.1	S2_03, S2_08 a S2_11 – Operações com ativos	3
4.2	S2_12 a S2_14 – Vulnerabilidades	3
5	Requisitos, validações e critérios de aceite	4
5.1	S2_04 a S2_07 – Como saber se está certo	4
6	Dicionários, JSON e testes	4
6.1	S2_15 – Dicionários como índice	4
6.2	S2_18 – Testes e correções	4
7	GitHub e roteiro da apresentação	5
7.1	S2_16, S2_17 e S2_19 – Entrega	5
8	Referências gerais	5

1 O que mudou da Sprint 1 para a Sprint 2

Na Sprint 1, os conceitos foram praticados em exercícios pequenos: entrada, saída, condições, listas, dicionários, JSON e Git. A Sprint 2 junta essas peças em um programa único. O foco deixa de ser “como usar um comando isolado” e passa a ser “como organizar uma funcionalidade completa sem tornar o código complicado”.

O programa continua sendo de terminal e usa apenas recursos básicos de Python: funções, classes simples, `Enum`, dicionários, listas, arquivo JSON e `try/except`. Não há banco de dados, interface gráfica, framework ou consulta automática à internet. Essa escolha é intencional: o problema é pequeno e precisa ser fácil de explicar.

2 Organização simples do código

O projeto foi separado por responsabilidade. Separar não significa criar muitas camadas: significa evitar que um mesmo arquivo faça tudo ao mesmo tempo.

- `app.py`: inicia o programa e informa o caminho do JSON.
- `interface.py`: contém `input()`, `print()` e o menu.
- `inventario.py`: contém regras de cadastro, consulta, atualização e exclusão.
- `modelos.py`: define `Ativo`, `Vulnerabilidade` e `TipoAtivo`.
- `persistencia.py`: lê e grava o arquivo JSON.
- `test_sprint2.py`: verifica automaticamente regras importantes.

Listing 1: Fluxo principal simplificado

```
ativos = carregar_ativos("dados/inventario.json")
executar_programa(ativos, "dados/inventario.json")
```

Aplicação no projeto. O menu não precisa conhecer os detalhes de JSON, e a função que salva JSON não precisa fazer perguntas ao usuário. Quando cada parte tem uma tarefa principal, fica mais fácil localizar erros e explicar o código.

3 Classes e Enum

3.1 S2_01 e S2_02 – Boas práticas, classes e funções

Uma classe é um molde para representar dados relacionados. Neste trabalho, `Ativo` reúne ID, nome, responsável, localização, tipo, criticidade, vulnerabilidades e histórico. `Vulnerabilidade` reúne CVE, CWE, CVSS, descrição, fonte, data, impacto, prioridade, tratamento, status e verificação.

Listing 2: Ideia de uma classe simples

```
class Ativo:
    def __init__(self, identificador, nome):
        self.id = identificador
        self.nome = nome
        self.vulnerabilidades = []
```

`self` representa o objeto que está sendo criado ou alterado. Assim, dois ativos podem existir ao mesmo tempo, cada um com seus próprios dados. Uma função como `cadastrar_ativo()` recebe os dados, verifica regras e cria um objeto. Ela não usa `input()` nem `print()`; isso fica na interface.

`TipoAtivo` é um `IntEnum`. Ele associa nomes claros a códigos inteiros, como `SERVIDOR = 2`. Isso atende ao requisito de possuir categorias padronizadas sem espalhar números misteriosos pelo código.

Card e referência. [S2_01 no Trello](#); [S2_02 no Trello](#).

4 CRUD de ativos e vulnerabilidades

4.1 S2_03, S2_08 a S2_11 – Operações com ativos

CRUD é uma sigla para quatro operações sobre dados: *Create* (criar), *Read* (consultar), *Update* (atualizar) e *Delete* (excluir). O dicionário `ativos` usa o ID como chave. Isso permite localizar um ativo diretamente.

Listing 3: Busca direta pela chave do dicionário

```
def consultar_por_id(ativos, identificador):  
    return ativos.get(identificador)
```

`.get()` devolve o valor encontrado ou `None` quando a chave não existe. Isso evita erro ao buscar um ID ausente. No cadastro, o programa rejeita ID duplicado. Na atualização, o ID não é aceito como campo modificável: nome, responsável, localização, tipo e criticidade podem mudar, mas a identidade do ativo é preservada. Na exclusão, a tela mostra o alvo e exige a palavra **SIM**; ao remover o ativo, sua lista de vulnerabilidades também desaparece junto.

4.2 S2_12 a S2_14 – Vulnerabilidades

Uma vulnerabilidade é uma fragilidade; ameaça é algo que pode explorar uma fragilidade; risco combina a possibilidade do problema com seu impacto. Por isso, o programa não usa apenas uma nota CVSS: ele também registra impacto no ativo, prioridade, tratamento, status e como a correção será verificada.

Cada vulnerabilidade pertence a um ativo. O par “ID do ativo + CVE” a identifica de forma simples: o mesmo CVE pode afetar ativos diferentes, mas não pode aparecer duas vezes no mesmo ativo. As quatro operações também existem para vulnerabilidades: cadastrar, consultar, atualizar e remover.

Listing 4: Validação simples do CVSS

```
nota = float(cvss)  
if not 0.0 <= nota <= 10.0:  
    raise DadosInvalidosError("CVSS deve estar entre 0.0 e 10.0.")
```

O `raise` interrompe apenas a operação inválida e a interface mostra uma mensagem compreensível. O menu continua aberto. O sistema também exige uma verificação antes de mudar o status para “Corrigida”, pois dizer que algo foi corrigido sem evidência não é suficiente.

Como referência de consulta externa, o registro [CVE-2021-44228 no NVD](#) descreve uma vulnerabilidade do Apache Log4j com CVSS 10.0. O programa não consulta o NVD automaticamente: o usuário registra manualmente fonte e data, mantendo a aplicação simples e funcionando sem internet.

Card e referência. [S2_03 no Trello](#); [S2_12 no Trello](#); [S2_13 no Trello](#); [S2_14 no Trello](#).

5 Requisitos, validações e critérios de aceite

5.1 S2_04 a S2_07 – Como saber se está certo

Um requisito diz o que o sistema deve fazer. Um critério de aceite diz como comprovar que ele fez corretamente. Por exemplo, “cadastrar ativo” é o requisito; “o ID precisa ser inteiro, positivo e único, e o ativo deve aparecer na busca” é o critério verificável.

- Campo obrigatório vazio: o programa pede novamente, sem fechar.
- ID repetido: o ativo antigo não é alterado e o novo não é cadastrado.
- Busca inexistente: aparece “Ativo não encontrado”, sem erro técnico.
- Exclusão cancelada: nada é removido.
- JSON inválido: o programa não sobrescreve o arquivo corrompido com uma base vazia.

Essas regras aparecem em funções de validação, como `validar_texto()`, `validar_cve()`, `validar_cwe()` e `validar_cvss()`. Quando uma regra falha, elas usam `DadosInvalidosError`; a interface captura essa exceção e apresenta uma mensagem curta.

Card e referência. [S2_04 no Trello](#); [S2_05 no Trello](#); [S2_06 no Trello](#); [S2_07 no Trello](#).

6 Dicionários, JSON e testes

6.1 S2_15 – Dicionários como índice

O dicionário é a estrutura central do inventário:

Listing 5: Estrutura principal na memória

```
ativos = {
    10: ativo_servidor,
    20: ativo_notebook
}
```

Em vez de percorrer todos os itens para achar o ID 10, o código usa `ativos.get(10)`. A chave é única, o formato dos objetos é uniforme e as alterações preservam a mesma chave. Ao salvar, os objetos viram dicionários simples; ao carregar, voltam a ser objetos `Ativo` e `Vulnerabilidade`.

6.2 S2_18 – Testes e correções

`test_sprint2.py` não é um segundo programa para o usuário usar. É uma lista de verificações automáticas feita com o módulo padrão `unittest`. Cada teste cria uma base pequena na memória, executa uma regra e compara o resultado esperado com o obtido.

Listing 6: Como executar os testes

```
python -m unittest -v test_sprint2.py
```

Os 14 testes verificam, entre outros casos, ID duplicado, tipo inválido, filtro sem alteração da base, atualização preservando ID, confirmação de exclusão, CVE duplicado, CVSS inválido, CRUD de vulnerabilidades e proteção contra JSON corrompido. Eles não usam dados reais, internet ou senhas.

7 GitHub e roteiro da apresentação

7.1 S2_16, S2_17 e S2_19 – Entrega

O código está no GitHub, na branch `main`. O histórico possui branches de funcionalidade, commits com mensagens explicativas e merges por Pull Request. A tag `trabalho-1-entrega` identifica a versão preparada para apresentação.

Para demonstrar em cinco minutos:

1. Execute `python app.py`.
2. Cadastre um ativo e uma vulnerabilidade associada.
3. Consulte o ativo e mostre que a vulnerabilidade está vinculada a ele.
4. Atualize o responsável ou o status da vulnerabilidade.
5. Tente repetir um ID ou informar CVSS inválido para mostrar a validação.
6. Remova uma vulnerabilidade, primeiro cancelando e depois confirmando.
7. Abra o GitHub, mostre `main`, as Pull Requests e a tag final.

Ao explicar, prefira frases concretas: “usei um dicionário porque o ID é único e a busca fica direta”; “usei JSON porque os dados continuam depois que o programa fecha”; “separei interface e regras para não misturar perguntas com validação”.

Card e referência. [S2_16 no Trello](#); [S2_17 no Trello](#); [S2_19 no Trello](#).

8 Referências gerais

- Cards S2_01 a S2_19 do quadro Trello da disciplina.
- [Documentação oficial do Python](#).
- [Documentação de unittest](#).
- [NVD - CVE-2021-44228](#).
- [Pro Git](#).